Google Skills

The solution:

Custom function tools

Write Python functions, get agent capabilities

Reference: ADK Docs - Function Tools

“Transforming a Python function into a tool is a straightforward way to integrate custom logic into your agents. When you assign a function to an agent’s tools list, the framework automatically wraps it as a FunctionTool.”

How it works:

1. You write: Standard Python function with your business logic
2. ADK inspects: Function name, docstring, parameters, type hints
3. ADK generates: Schema describing the tool for the LLM
4. Agent uses: Tool based on schema and docstring

Simple example:

Python

```
from google.adk.agents import LlmAgent

# Step 1: Write a Python function
def calculate_shipping_cost(weight_kg: float, country: str) -> dict:
    """Calculates shipping cost based on package weight and destination."""
    rates = {"usa": 10, "canada": 12, "uk": 15}

    if country.lower() not in rates:
        return {"status": "error", "error_message": f"We don't ship to
{country}"}

    cost = weight_kg * rates[country.lower()]
    return {"status": "success", "cost_usd": cost}

# Step 2: Add function to agent's tools list
agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="You help customers with shipping cost estimates.",
    tools=[calculate_shipping_cost]  # ADK automatically wraps as FunctionTool
)
```

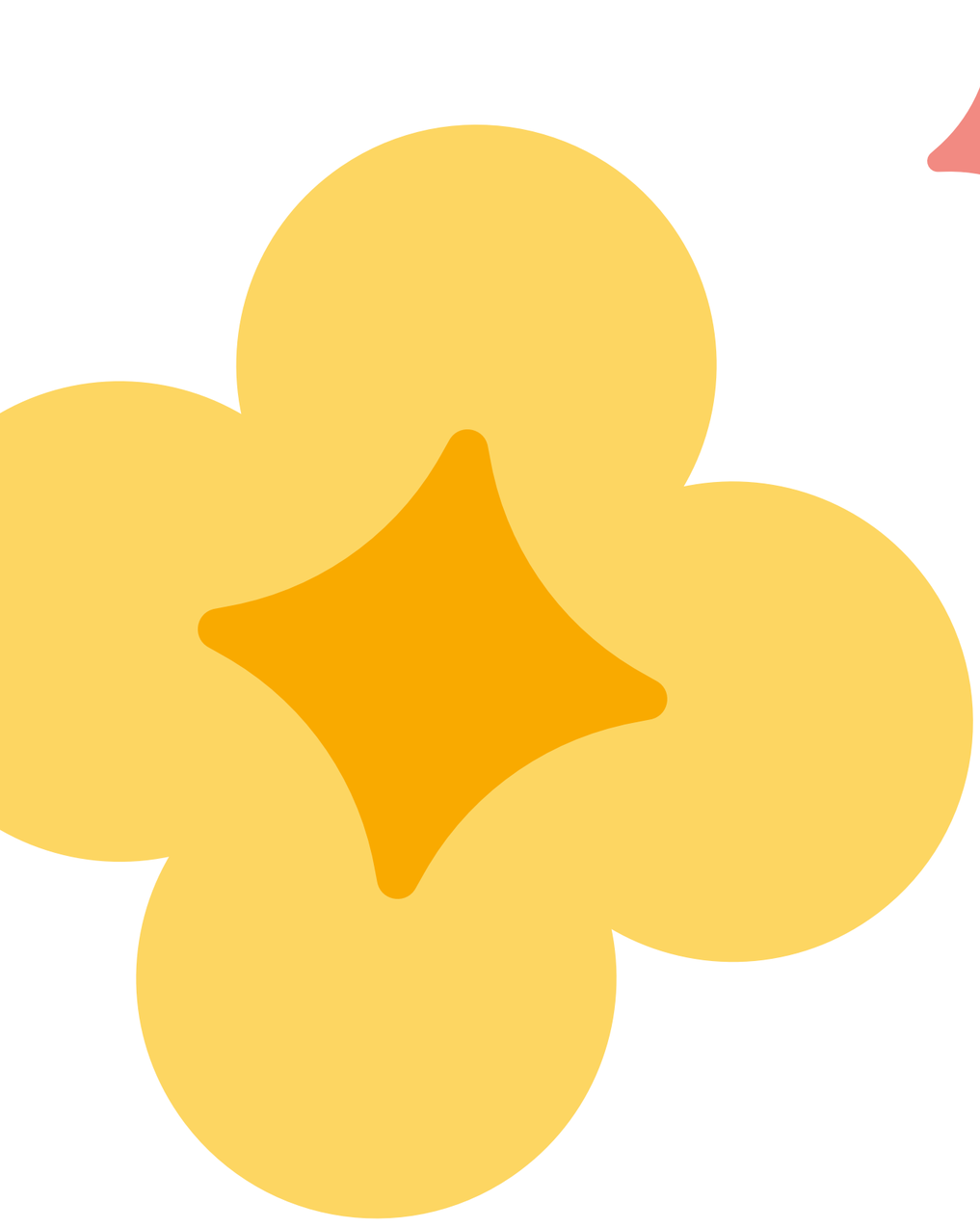
What this enables:

- **Integrate your business logic** directly into agent capabilities
- **Access your proprietary systems** through Python code
- **Implement domain-specific calculations** unique to your business
- **Full control** over implementation and behavior
- **Automatic schema generation** from your function signature

How custom tools integrate with agents:

```
None
sequenceDiagram
    participant User
    participant Agent
    participant Tool as Custom Tool
    participant System as Your System/API

    User->>Agent: "Calculate shipping to Canada"
    Agent->>Agent: Analyze request & select tool
    Agent->>Tool: calculate_shipping_cost(weight=5, country="Canada")
    Tool->>System: Look up rates in database
    System-->>Tool: Rate data
    Tool->>Tool: Calculate: 5kg × $12/kg
    Tool-->>Agent: {"status": "success", "cost_usd": 60}
    Agent-->>User: "Shipping cost is $60 USD"

    Note over Agent,Tool: LLM decides WHEN to call<br/>Tool executes WHAT to do
```

Custom tools enable agents to access your business logic and systems



Core concepts

1. Function signatures matter

Reference: ADK Docs - Function Tools

From ADK documentation:

“The function signature, including parameter types and return type, is crucial. ADK uses this information to generate the schema that the LLM sees.”

Critical elements:

1. Function Name (descriptive)

The LLM uses the function name to understand what the tool does.

```python
# ✅ Good: Descriptive, verb-noun pattern
def get_shipping_cost(weight: float, destination: str) -> dict:
    """Retrieves shipping cost for a package."""
    pass

def calculate_loyalty_points(purchase_amount: float) -> dict:
    """Calculates loyalty points earned from a purchase."""
    pass

def search_available_flights(destination: str, date: str) -> dict:
    """Searches for available flights to a destination."""
    pass

# ❌ Bad: Generic, unclear names
def process(data: float) -> dict:  # Process what?
    pass

def do_stuff(x: str) -> dict:  # What stuff?
    pass

def handler(amount: float) -> dict:  # Handle what?
    pass
```

2. Type Hints (required)

Type hints tell ADK what types the LLM should provide.

```python
# ✅ Good: Type hints for all parameters and return
def lookup_order(order_id: str, user_id: int) -> dict:
    """Looks up order information."""
    pass

def calculate_discount(price: float, customer_tier: str) -> dict:
    """Calculates discount based on customer tier."""
    pass

# ❌ Bad: No type hints - LLM won't know what types to provide
def lookup_order(order_id, user_id):  # What types are these?
    pass

def calculate_discount(price, customer_tier):  # Unknown types
    pass
```

3. Parameter types

Use JSON-serializable types that LLMs understand:

- ✅ **Supported:** str, int, float, bool, list, dict
- ❌ **Avoid:** Complex custom classes, objects, file handles

```python
# ✅ Good: Simple, JSON-serializable types
def book_flight(
    destination: str,
    departure_date: str,
    passengers: int
) -> dict:
    pass

# ❌ Bad: Complex custom types
from datetime import datetime
from custom_models import Customer

def book_flight(
    destination: str,
    departure_date: datetime,  # Not JSON-serializable
    customer: Customer  # Custom class
) -> dict:
    pass
```

4. Required vs optional parameters

From ADK documentation:

“Do not set default values for parameters.
Default values are not reliably supported or used by the underlying models.”

Recommendation: Use all required parameters.

```python
# ✅ Recommended: All parameters required
def book_flight(destination: str, date: str, passengers: int) -> dict:
    """Books a flight."""
    pass

# ⚠️ Not recommended: Default values may not work reliably
def search_flights(
    destination: str,
    max_price: float = 1000.0,  # Default may be ignored
    class_type: str = "economy"  # Default may be ignored
) -> dict:
    """Searches for flights."""
    pass
```

2. Docstrings are critical

Reference: ADK Docs - Function Tools

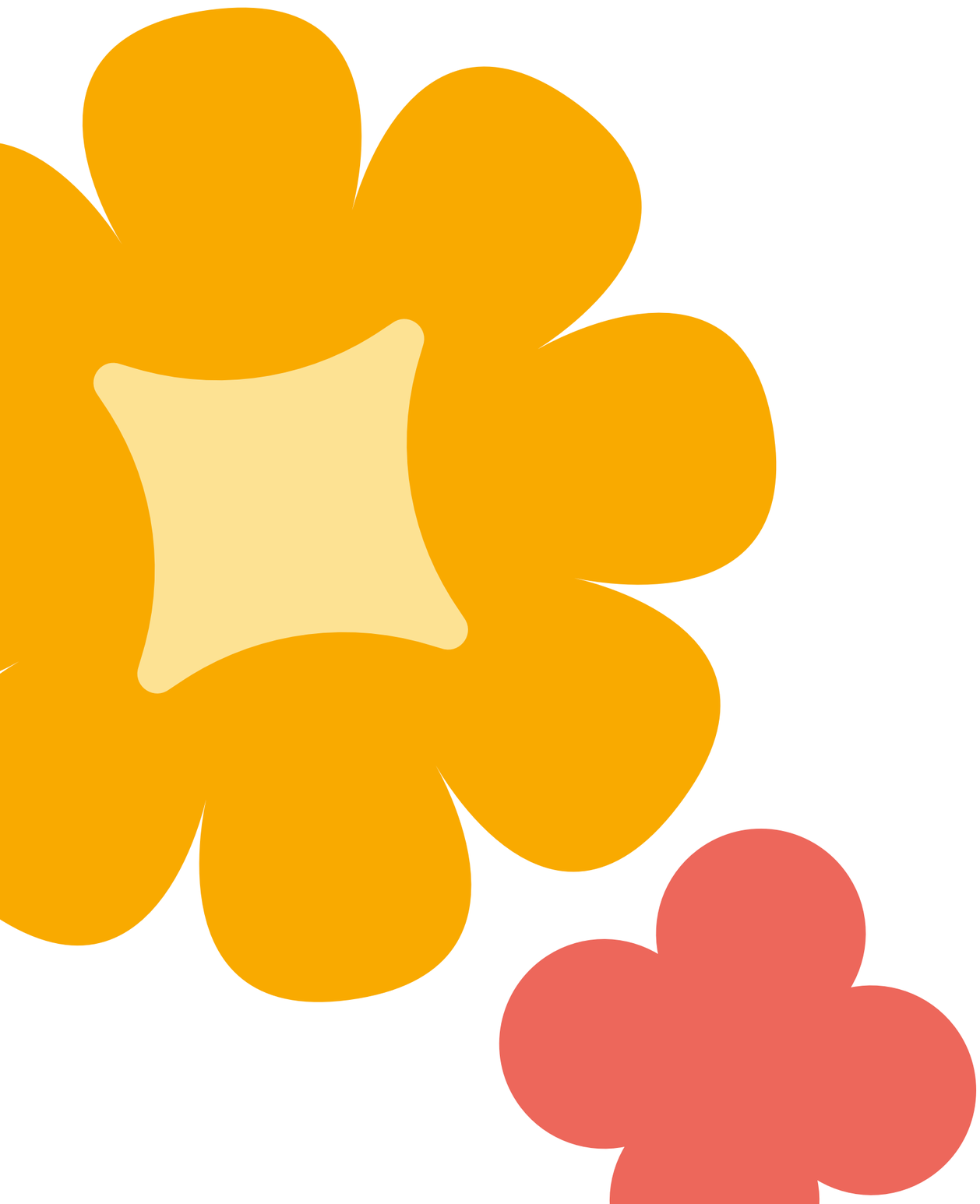
From ADK documentation:

“The docstring of your function serves as the tool’s description and is sent to the LLM. Therefore, a well-written and comprehensive docstring is crucial for the LLM to understand how to use the tool effectively.”

Docstring template:

Python

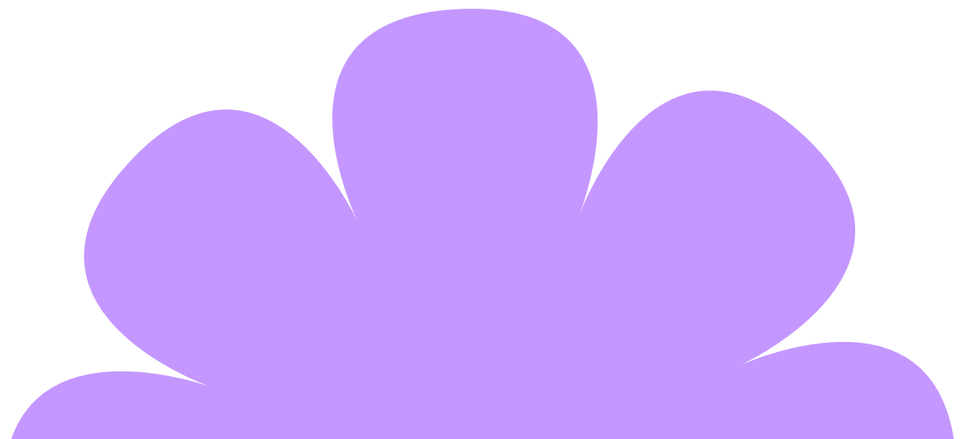
```
def tool_name(param1: type1, param2: type2) -> dict:
    """[One-line summary of what this tool does]

    [Optional: Additional context about when to use this tool]

    Args:
        param1 (type1): [Description of param1]
        param2 (type2): [Description of param2]

    Returns:
        dict: [Description of return structure]
            On success: {'status': 'success', 'key': value}
            On error: {'status': 'error', 'error_message': 'explanation'}
    """
    # Implementation
    pass
```



Complete example:

Python

```
def calculate_shipping_cost(weight_kg: float, country: str) -> dict:
    """Calculates shipping cost based on package weight and destination country.

    Use this tool when a customer asks about shipping costs for their order.

    Args:
        weight_kg (float): The package weight in kilograms.
        country (str): The destination country name.

    Returns:
        dict: Shipping cost information.
            On success: {'status': 'success', 'cost_usd': 25.50, 'delivery_days':
5}
            On error: {'status': 'error', 'error_message': 'Country not
supported'}
    """
    shipping_rates = {
        "usa": {"rate_per_kg": 10, "days": 3},
        "canada": {"rate_per_kg": 12, "days": 5},
        "uk": {"rate_per_kg": 15, "days": 7},
    }

    country_key = country.lower()
    if country_key not in shipping_rates:
        return {
            "status": "error",
            "error_message": f"Shipping to {country} is not currently supported."
        }

    rate_info = shipping_rates[country_key]
    cost = weight_kg * rate_info["rate_per_kg"]

    return {
        "status": "success",
        "cost_usd": round(cost, 2),
        "delivery_days": rate_info["days"],
        "destination": country
    }
```



What the LLM sees:

```
None
Tool: calculate_shipping_cost

Description:
  Calculates shipping cost based on package weight and destination country.
  Use this tool when a customer asks about shipping costs for their order.

Parameters:
  - weight_kg (float): The package weight in kilograms
  - country (str): The destination country name

Returns:
  Dictionary with status and shipping information
```

The LLM uses this to decide:

- **When to call:** “Customer asks about shipping costs”
- **What parameters:** weight_kg (float), country (str)
- **What to expect:** Dictionary with cost_usd and delivery_days

How ADK converts your function to LLM schema:

```
None
graph TB
    A[Your Python Function] --> B[Function Name]
    A --> C[Type Hints]
    A --> D[Docstring]
    A --> E[Return Type]

    B --> F[LLM Schema Generation]
    C --> F
    D --> F
    E --> F

    F --> G[Tool Name]
    F --> H[Tool Description]
    F --> I[Parameter Types]
    F --> J[Expected Output]

    G --> K[LLM Tool Selection]
    H --> K
    I --> K
    J --> K

    K --> L[Agent calls tool<br/>with correct parameters]

    style F fill:#ffeb99
    style K fill:#e1f5ff
```

ADK automatically generates LLM schema from your function metadata—no manual configuration needed

3. Return dictionaries with status

Reference: ADK Docs - Function Tools

From ADK documentation:

“The preferred return type for a Function Tool is a dictionary in Python...
It's a highly recommended practice to include a status key (e.g., 'success', 'error', 'pending')
to clearly indicate the outcome of the tool execution for the model.”

Pattern:

```python
# Success case
return {
    "status": "success",
    "data_key": value,
    "another_key": another_value
}

# Error case
return {
    "status": "error",
    "error_message": "Human-readable explanation of what went wrong"
}
```

Why this matters:

- **LLM understanding:** Clear status helps LLM know if tool succeeded
- **Next action:** LLM decides what to do based on status (continue, retry, apologize)
- **Error handling:** Error messages guide LLM's response to user
- **Structured data:** Consistent format makes LLM responses more reliable

Example:

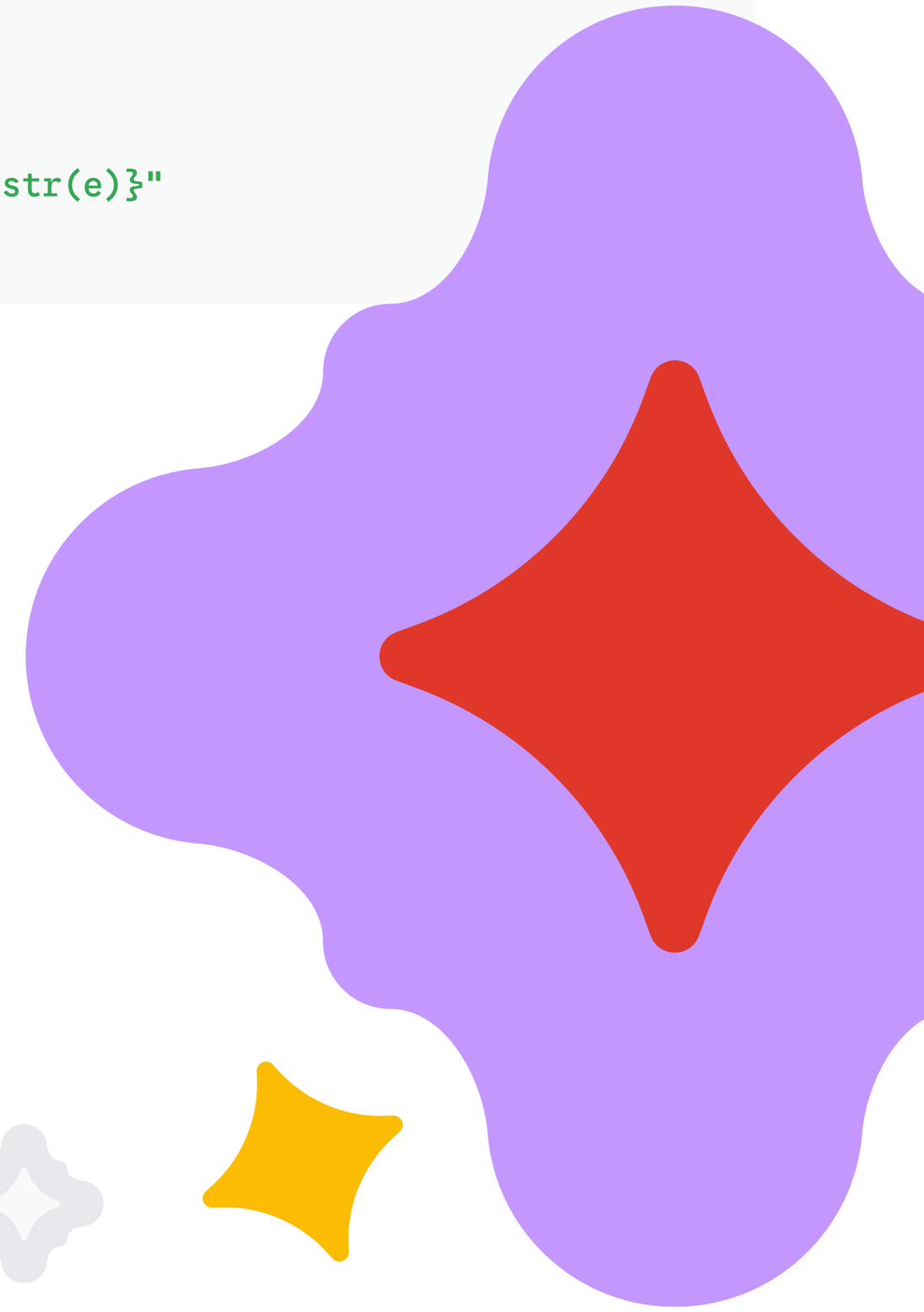
```python
def lookup_product(product_id: str) -> dict:
    """Looks up product information by ID.

    Args:
        product_id (str): The product ID to look up.

    Returns:
        dict: Product information.
            On success: {'status': 'success', 'product': {...}}
            On error: {'status': 'error', 'error_message': 'explanation'}
    """
    # Simulated database
    products = {
        "PROD001": {"name": "Widget Pro", "price": 99.99, "in_stock": True},
        "PROD002": {"name": "Gadget Plus", "price": 149.99, "in_stock": False}
    }

    # Success case
    if product_id in products:
        return {
            "status": "success",
            "product_id": product_id,
            "product": products[product_id]
        }

    # Error case
    return {
        "status": "error",
        "error_message": f"Product {product_id} not found in database."
    }
```

How the LLM uses this:

```
User: "Tell me about product PROD999"

Agent calls: lookup_product("PROD999")

Tool returns: {"status": "error", "error_message": "Product PROD999 not found in
database."}

LLM sees status="error" and responds:
"I apologize, but I couldn't find product PROD999 in our system. Could you verify
the product ID?"
```

Key pattern:

```python
def your_tool(params) -> dict:
    """Your docstring."""

    # Validate input
    if validation_fails:
        return {
            "status": "error",
            "error_message": "Clear explanation for the user"
        }

    # Perform operation
    try:
        result = do_something()
        return {
            "status": "success",
            "result_key": result,
            "additional_info": other_data
        }
    except Exception as e:
        return {
            "status": "error",
            "error_message": f"Operation failed: {str(e)}"
        }
```



4. Using state in custom tools (preview)

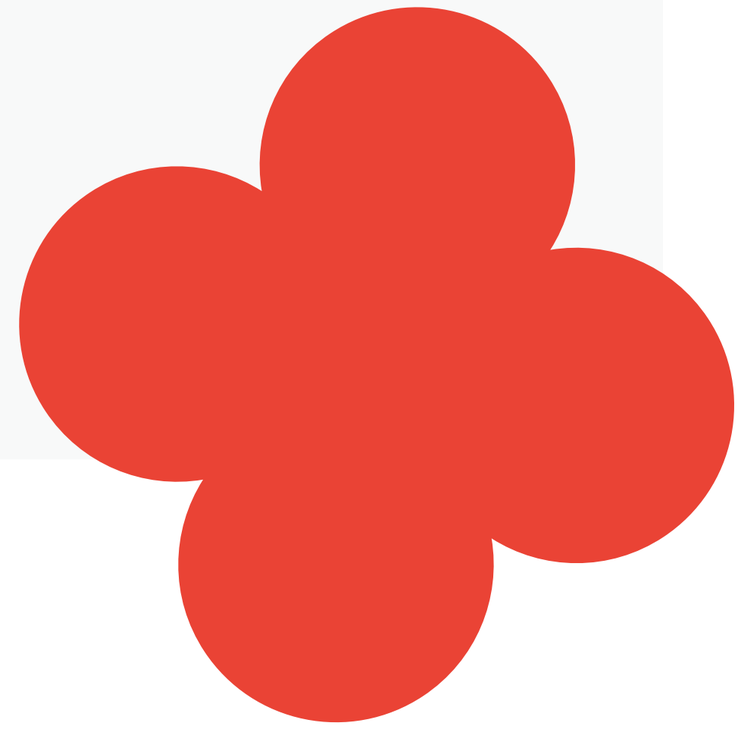
Connection to Lesson 4: Managing state and memory

Custom tools can access and modify session state to provide personalized, context-aware behavior. While the mechanics of tool_context are covered in future lessons, here's what's possible:

Example: Tool that uses state for personalization

Python

```
def recommend_products(category: str, ctx) -> dict:
    """Recommends products based on user preferences from state.

    Args:
        category: Product category to search
        ctx: Tool context (provides access to session state)

    Returns:
        dict: Recommended products with personalization info

    Note: The ctx parameter (tool_context) is covered in Lesson 6.
    """
    # Access user preferences from state
    user_tier = ctx.session.state.get('user:tier', 'standard')
    user_preferences = ctx.session.state.get('user:preferences', {})

    # Personalize recommendations based on tier and preferences
    if user_tier == 'premium':
        products = get_premium_products(category, user_preferences)
        discount = 0.15
    else:
        products = get_standard_products(category)
        discount = 0.05

    return {
        "status": "success",
        "products": products,
        "tier": user_tier,
        "discount_applied": discount
    }
```



When to use state in tools:

✅ Personalization based on user preferences

```python
# Access user:language to provide localized results
language = ctx.session.state.get('user:language', 'en')
```

✅ Context from previous tool calls

```python
# Check if previous operation completed
prev_status = ctx.session.state.get('temp:last_operation_status')
```

✅ Multi-step workflows requiring intermediate data

```python
# Save intermediate results for next tool
ctx.session.state['temp:calculation_result'] = computed_value
```

✅ User-specific configuration

```python
# Use user's preferred units
units = ctx.session.state.get('user:preferred_units', 'imperial')
```

State namespaces in tools (from lesson 4):

- `temp:` - Temporary data, discarded after turn
- No prefix - Session data, discarded after conversation ends
- `user:` - User preferences, persists across all sessions
- `app:` - Global configuration, shared by all users

Example: Multi-step workflow with state

Python

```
def start_order_process(product_id: str, ctx) -> dict:
    """Initiates order process and saves to state."""
    # Save intermediate data to session state
    ctx.session.state['temp:current_order'] = {
        'product_id': product_id,
        'step': 'initiated',
        'timestamp': time.time()
    }
    return {"status": "success", "message": "Order started"}

def add_shipping_address(address: dict, ctx) -> dict:
    """Adds shipping address to ongoing order."""
    # Retrieve intermediate data from state
    order_data = ctx.session.state.get('temp:current_order')
    if not order_data:
        return {"status": "error", "error_message": "No active order"}

    # Update order in state
    order_data['shipping_address'] = address
    order_data['step'] = 'address_added'
    ctx.session.state['temp:current_order'] = order_data

    return {"status": "success", "message": "Address added"}

def finalize_order(ctx) -> dict:
    """Completes order using data from state."""
    # Retrieve complete order data
    order_data = ctx.session.state.get('temp:current_order')
    if not order_data or order_data['step'] != 'address_added':
        return {"status": "error", "error_message": "Order not ready"}

    # Process order (API call, database, etc.)
    order_id = process_order_in_system(order_data)

    # Clean up temp state
    del ctx.session.state['temp:current_order']

    return {"status": "success", "order_id": order_id}
```

What this demonstrates:

- Tools can read user preferences from `user:` namespace
- Tools can save/retrieve intermediate data with `temp:` namespace
- Multiple tools coordinate through shared state
- State enables complex, stateful workflows

Preview: Full integration of tools with state management, including the ctx parameter and tool_context, is covered in future lessons on callbacks and safety guardrails.

5. Multiple tools working together

Pattern: List multiple functions in tools parameter

Agents can use multiple tools to accomplish complex tasks:

Python

```
from google.adk.agents import LlmAgent

def check_inventory(product_id: str) -> dict:
    """Checks if a product is in stock."""
    # Implementation
    return {"status": "success", "in_stock": True, "quantity": 5}

def get_product_price(product_id: str) -> dict:
    """Gets the current price of a product."""
    # Implementation
    return {"status": "success", "price_usd": 99.99}

def calculate_shipping(weight_kg: float, country: str) -> dict:
    """Calculates shipping cost."""
    # Implementation
    return {"status": "success", "cost_usd": 25.50}

def calculate_total(product_price: float, shipping_cost: float) -> dict:
    """Calculates total cost including product and shipping."""
    total = product_price + shipping_cost
    return {
        "status": "success",
        "subtotal": product_price,
        "shipping": shipping_cost,
        "total": total
    }

# Agent with multiple tools
agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="""
    When a user asks about total cost:
    1. First check inventory with check_inventory
    2. If in stock, get price with get_product_price
    3. Calculate shipping with calculate_shipping
    4. Use calculate_total to provide the final price

    Present the breakdown clearly to the customer.
    """,
    tools=[check_inventory, get_product_price, calculate_shipping,
calculate_total]
)
```

How the agent orchestrates:

```
None
User: "What's the total cost for product PROD001 shipped to Canada?"

Agent reasoning:
1. Calls check_inventory("PROD001") → in_stock: True
2. Calls get_product_price("PROD001") → price: $99.99
3. Calls calculate_shipping(weight, "Canada") → shipping: $25.50
4. Calls calculate_total(99.99, 25.50) → total: $125.49

Agent responds: "Product PROD001 is available! The price is $99.99,
shipping to Canada is $25.50, for a total of $125.49."
```

Key points:

- **Sequential calls:** Agent calls tools in logical order
- **Result chaining:** Uses output from one tool as context for next
- **Automatic orchestration:** Agent decides which tools to use and when
- **Natural synthesis:** Combines all results into coherent response

---

Hands-on example

Building a travel agent with custom tools

What you'll build:

A travel agent that helps users plan trips by searching for flights, hotels, and calculating budgets

Step 1: Create the project

```
Shell
adk create travel_agent
cd travel_agent
```

What this does:

- Creates new ADK project directory
- Sets up basic agent structure
- Creates `agent.py` with default code

Step 2: Write the agent

Replace the contents of agent.py with:

Python

```
"""
Travel Agent with Custom Function Tools
Demonstrates multiple custom tools working together.

Reference: https://google.github.io/adk-docs/tools-custom/function-tools/
"""

from google.adk.agents import LlmAgent

# Tool 1: Search flights
def search_flights(destination: str, departure_date: str) -> dict:
    """Searches for available flights to a destination on a specific date.

    Use this tool when a customer wants to know flight options.

    Args:
        destination (str): The destination city (e.g., "Paris", "Tokyo").
        departure_date (str): Departure date in YYYY-MM-DD format.

    Returns:
        dict: Flight search results.
            On success: {'status': 'success', 'flights': [...], 'count': N}
            On error: {'status': 'error', 'error_message': 'explanation'}
    """
    # Simulated flight data
    available_flights = {
        "paris": [
            {"flight_number": "AF123", "price_usd": 450, "duration_hours": 8},
            {"flight_number": "BA456", "price_usd": 480, "duration_hours": 7.5},
        ],
        "tokyo": [
            {"flight_number": "JL789", "price_usd": 850, "duration_hours": 13},
            {"flight_number": "ANA101", "price_usd": 820, "duration_hours":
12.5},
        ],
    }

    dest_key = destination.lower()
    if dest_key not in available_flights:
        return {
            "status": "error",
            "error_message": f"No flights found to {destination}. Try Paris or
Tokyo."
        }
```

```
    return {
        "status": "success",
        "destination": destination,
        "departure_date": departure_date,
        "flights": available_flights[dest_key],
        "count": len(available_flights[dest_key])
    }

# Tool 2: Search hotels
def search_hotels(city: str, check_in_date: str) -> dict:
    """Searches for available hotels in a city for a specific check-in date.

    Use this tool when a customer needs accommodation.

    Args:
        city (str): The city name (e.g., "Paris", "Tokyo").
        check_in_date (str): Check-in date in YYYY-MM-DD format.

    Returns:
        dict: Hotel search results.
            On success: {'status': 'success', 'hotels': [...], 'count': N}
            On error: {'status': 'error', 'error_message': 'explanation'}
    """
    # Simulated hotel data
    available_hotels = {
        "paris": [
            {"name": "Hotel Eiffel", "price_per_night_usd": 150, "rating": 4.5},
            {"name": "Louvre Inn", "price_per_night_usd": 120, "rating": 4.2},
        ],
        "tokyo": [
            {"name": "Shibuya Grand", "price_per_night_usd": 180, "rating": 4.7},
            {"name": "Tokyo Bay Hotel", "price_per_night_usd": 140, "rating":
4.3},
        ],
    }

    city_key = city.lower()
    if city_key not in available_hotels:
        return {
            "status": "error",
            "error_message": f"No hotels found in {city}. Try Paris or Tokyo."
        }

    return {
        "status": "success",
        "city": city,
        "check_in_date": check_in_date,
        "hotels": available_hotels[city_key],
        "count": len(available_hotels[city_key])
    }
```

```
# Tool 3: Calculate trip budget
def calculate_trip_budget(flight_price: float, hotel_price: float, num_nights:
int) -> dict:
    """Calculates total trip budget including flights and accommodation.

    Use this after finding flight and hotel prices to give the customer a total
estimate.

    Args:
        flight_price (float): Round-trip flight cost in USD.
        hotel_price (float): Hotel cost per night in USD.
        num_nights (int): Number of nights staying.

    Returns:
        dict: Budget breakdown.
              Always returns: {'status': 'success', 'total_usd': X, 'breakdown':
{...}}
    """
    hotel_total = hotel_price * num_nights
    total = flight_price + hotel_total

    return {
        "status": "success",
        "total_usd": round(total, 2),
        "breakdown": {
            "flight_cost": flight_price,
            "hotel_cost_per_night": hotel_price,
            "num_nights": num_nights,
            "hotel_total": round(hotel_total, 2)
        }
    }

# Create travel agent with all three tools
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='travel_agent',
    description='Helps users plan trips by finding flights and hotels.',
    instruction="""
    You are a helpful travel agent assistant.

    Your capabilities:
    - Search for flights using search_flights(destination, departure_date)
    - Search for hotels using search_hotels(city, check_in_date)
    - Calculate trip budgets using calculate_trip_budget(flight_price,
hotel_price, num_nights)

    When helping users:
    1. If they ask about flights, use search_flights
    2. If they ask about hotels, use search_hotels
    3. If they want a full trip estimate, use both search tools, then
calculate_trip_budget
```

```
    4. Always present options clearly with prices
     5. If a tool returns an error, apologize and suggest available destinations
(Paris or Tokyo)

     Be friendly and help users plan their perfect trip!
     """,
     tools=[search_flights, search_hotels, calculate_trip_budget]
)
```

Code walkthrough:

Lines 10–50: search_flights tool

- Descriptive name: `search_flights` (verb-noun pattern)
- Type hints: `str`, `str` → `dict`
- Comprehensive docstring explaining what, when, args, returns
- Returns structured dict with `status` key
- Error handling for unsupported destinations

Lines 52–91: search_hotels tool

- Similar pattern to search_flights
- Different data structure (hotels vs flights)
- Consistent error handling with status and error_message

Lines 93–116: calculate_trip_budget tool

- Takes output from other tools as input (flight_price, hotel_price)
- Performs calculation
- Returns detailed breakdown
- Always succeeds (status: success)

Lines 118–142: Agent with multiple tools

- All three tools in tools list
- Instruction guides when to use each tool
- Explains how to handle errors
- Guides sequential usage for complex requests

Step 3: Run and test

Shell

```
adk web
```

Navigate to `http://localhost:8000` in your browser.

Step 4: Test scenarios

Test 1: Flight search

You: `I want to fly to Paris on 2025-12-15`

Expected behavior:

- Agent calls `search_flights(destination="Paris", departure_date="2025-12-15")`
- Tool returns 2 flight options
- Agent presents options with flight numbers, prices, and durations

What to notice:

- Agent automatically extracted destination and date from natural language
- Tool returned structured data with multiple flight options
- Agent formatted results naturally for the user

Test 2: Complete trip planning

You: `Plan a 3-night trip to Tokyo starting 2025-12-20`

Expected behavior:

1. Agent calls `search_flights(destination="Tokyo", departure_date="2025-12-20")`
2. Agent calls `search_hotels(city="Tokyo", check_in_date="2025-12-20")`
3. Agent picks reasonable flight and hotel from options
4. Agent calls `calculate_trip_budget(flight_price, hotel_price, 3)`
5. Agent presents complete trip plan with total cost breakdown

What to notice:

- Agent orchestrated 3 different tool calls
- Tools worked together sequentially
- Final response synthesizes all information
- Natural language presentation of structured data

Test 3: Error handling

You: `Find hotels in London`

Expected behavior:

- Agent calls `search_hotels(city="London", check_in_date=...)`
- Tool returns error: `{"status": "error", "error_message": "No hotels found in London. Try Paris or Tokyo."}`
- Agent apologizes politely
- Agent suggests available destinations

What to notice:

- Error handling is graceful
- Agent doesn't hallucinate hotel data for unsupported city
- Clear guidance on what destinations are available
- Professional tone maintained even with errors

Test 4: Budget calculation only

You: `Plan a 3-night trip to Tokyo starting 2025-12-20`

Expected behavior:

- Agent calls `calculate_trip_budget(flight_price=450, hotel_price=150, num_nights=4)`
- Tool returns breakdown
- Agent presents: Flight $450 + Hotel $600 (4 nights × $150) = Total $1,050

What to notice:

- Agent selected the right tool for the task
- Didn't unnecessarily call search tools
- Tool focused on one clear task (calculation)
- Breakdown helps user understand the total

Key takeaways

Function tool essentials:

- **Descriptive function names:** Use verb-noun pattern (get_, calculate_, search_*)
- **Type hints required:** ADK uses these to generate schema for the LLM
- **Comprehensive docstrings:** Explain what the tool does, when to use it, args, and returns
- **Return dictionaries:** Always include status key for LLM comprehension

Best Practice Pattern:

Python

```
def tool_name(param: type) -> dict:
    """Clear one-line summary.

    Additional context about when to use this tool.

    Args:
        param (type): Description of parameter.

    Returns:
        dict: Description of return.
            On success: {'status': 'success', 'data': value}
            On error: {'status': 'error', 'error_message': 'explanation'}
    """
    # Validate input
    if error_condition:
        return {"status": "error", "error_message": "Clear explanation"}

    # Perform operation
    return {"status": "success", "result_key": computed_value}
```

Multiple Tools:

- **List all functions:** `tools=[tool1, tool2, tool3]`
- **Agent selects automatically:** Based on context and docstrings
- **Sequential usage:** Tools can build on each other's results
- **Reference in instructions:** Guide LLM on when to use each tool

Schema Generation:

ADK automatically creates tool schema from:

1. Function name
2. Parameter names and type hints
3. Docstring
4. Return type

When to Use Custom Tools:

- Business-specific calculations (shipping costs, discounts, taxes)
- Proprietary database lookups (orders, inventory, customers)
- Custom API integrations (your internal services)
- Domain-specific logic (booking systems, pricing rules)
- Any task unique to your application

Key Differences:

Aspect	Built-in Tools	Custom Function Tools
Setup	Import and use	Write function implementation
Maintenance	ADK team maintains	You maintain
Customization	Limited to tool features	Full control
Use for	Common capabilities	Business-specific logic